



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251229
Nama Lengkap	Ignatius Harya Nugraha
Minggu ke / Materi	13 / Tuple

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pengertian Tuple

Tuple adalah struktur data di Python yang mirip dengan list, namun memiliki perbedaan mendasar yaitu sifatnya yang immutable (tidak dapat diubah). Setelah tuple dibuat, elemen-elemennya tidak dapat dimodifikasi, ditambah, atau dihapus. Tuple menggunakan tanda kurung () untuk mendefinisikannya, berbeda dengan list yang menggunakan tanda kurung siku [].

Sifat immutable membuat tuple lebih efisien dalam penggunaan memori dan lebih cepat diakses dibanding list. Tuple juga bersifat hashable, artinya dapat digunakan sebagai key dalam dictionary atau sebagai elemen dalam set, sedangkan list tidak bisa karena bersifat mutable.

Cara membuat Tuple

Untuk membuat tuple di python ada beberapa cara yaitu:

```
# Cara membuat tuple
t1 = ('a', 'b', 'c', 'd', 'e')
t2 = 'a', 'b', 'c', 'd', 'e' # tanpa kurung juga valid

# Tuple dengan satu elemen (perlu koma!)
t3 = ('a',) # ini tuple
t4 = ('a') # ini STRING, bukan tuple

# Tuple kosong
t5 = tuple()

# Membuat tuple dari string
t6 = tuple('Python')
print(t6) # ('P', 'y', 't', 'h', 'o', 'n')
```

Perbedaan utama tuple dan list adalah tuple immutable (tidak bisa diubah) sedangkan list mutable (bisa diubah) lalu tuple menggunakan () sedangkan list menggunakan [] jika tuple bisa jadi key dictionary, list tidak bisa dalam pengaksesan tuple lebih cepat dan hemat memori dibanding list dengan jumlah elemen sama.

Operasi dasar Tuple

Akses Elemen dan Slicing

Seperti list, tuple menggunakan indeks untuk mengakses elemen. Indeks dimulai dari 0 untuk elemen pertama, dan -1 untuk elemen terakhir. Operasi slicing juga dapat dilakukan untuk mengambil sebagian elemen tuple.

```
t = ('a', 'b', 'c', 'd', 'e')

# Akses elemen
print(t[0])    # 'a'
print(t[-1])   # 'e'

# Slicing
print(t[1:3])  # ('b', 'c')
print(t[:2])   # ('a', 'b')
print(t[2:])   # ('c', 'd', 'e')

# Tuple immutable - tidak bisa diubah
# t[0] = 'A'   # ERROR: TypeError

# Tapi bisa diganti keseluruhan
t = ('A',) + t[1:]
print(t)       # ('A', 'b', 'c', 'd', 'e')
```

Operasi Lainnya

Tuple mendukung berbagai operasi seperti concatenation (penggabungan), repetition (pengulangan), membership test (cek keanggotaan), dan iteration (perulangan). Fungsi len(), min(), max(), dan sum() juga dapat digunakan pada tuple yang berisi elemen numerik.

```
# Concatenation
t1 = (1, 2, 3)
t2 = (4, 5, 6)
t3 = t1 + t2
print(t3)    # (1, 2, 3, 4, 5, 6)

# Repetition
t4 = ('ha',) * 3
print(t4)    # ('ha', 'ha', 'ha')
```

```
# Membership
print(3 in t1)      # True
print(10 in t1)     # False

# Panjang tuple
print(len(t1))      # 3
```

Membandingkan dan Mengurutkan Tuple

Tuple dapat dibandingkan menggunakan operator perbandingan (<, >, ==, !=, <=, >=). Python membandingkan tuple secara leksikografis, yaitu membandingkan elemen pertama dari kedua tuple, jika sama baru membandingkan elemen kedua, dan seterusnya. Prinsip ini berlaku juga saat mengurutkan list of tuples.

```
# Perbandingan tuple
print((0, 1, 2) < (0, 3, 4))
print((0, 1, 2000000) < (0, 3, 4))
print(('a', 'b') < ('a', 'c'))

# Mengurutkan list of tuples
data = [(3, 'tiga'), (1, 'satu'), (2, 'dua')]
data.sort()
print(data) # [(1, 'satu'), (2, 'dua'), (3, 'tiga')]

# Urutan terbalik (descending)
data.sort(reverse=True)
print(data) # [(3, 'tiga'), (2, 'dua'), (1, 'satu')]
```

Tuple Assignment (Penugasan Tuple)

Python memiliki fitur unik yaitu tuple assignment, yang memungkinkan kita menetapkan beberapa variabel sekaligus dalam satu baris. Fitur ini sangat berguna untuk unpacking values, swapping variables, dan memproses data terstruktur seperti hasil dari fungsi yang mengembalikan beberapa nilai.

```
x, y = 10, 20
print(x) # 10
print(y) # 20
```

```

# Unpacking list/tuple
data = ['Alice', 25, 'Jakarta']
nama, umur, kota = data
print(nama) # Alice

# Swap variabel (tanpa variabel temporary!)
a, b = 5, 10
a, b = b, a
print(a, b) # 10 5

# Split dan assign
email = 'user@domain.com'
username, domain = email.split('@')
print(username) # user
print(domain) # domain.com

```

Syarat tuple assignment: jumlah variabel di kiri harus sama dengan jumlah elemen di kanan. Jika tidak sama, akan muncul ValueError. Tuple assignment membuat kode lebih ringkas dan mudah dibaca, terutama saat bekerja dengan fungsi yang mengembalikan multiple values.

Dictionary dan Tuple

Method items() - Mendapatkan Key-Value Pairs

Dictionary memiliki method items() yang mengembalikan list of tuples, di mana setiap tuple berisi pasangan (key, value). Ini sangat berguna untuk iterasi dictionary atau mengurutkan dictionary berdasarkan key atau value.

```

# Dictionary to list of tuples
d = {'a': 10, 'b': 1, 'c': 22}
t = list(d.items())
print(t)
# [('a', 10), ('b', 1), ('c', 22)]

# Urutkan berdasarkan key
t.sort()
print(t)
# [('a', 10), ('b', 1), ('c', 22)]

```

Multi-Assignment dengan Dictionary

Kombinasi `items()`, tuple assignment, dan `for` loop menghasilkan cara yang elegan untuk iterasi dictionary. Kita dapat mengakses key dan value secara bersamaan dalam satu loop.

```
# Iterasi dictionary dengan tuple assignment
d = {'a': 10, 'b': 1, 'c': 22}

for key, val in d.items():
    print(f'{key}: {val}')
# a: 10
# c: 22
# b: 1
```

Mengurutkan Dictionary Berdasarkan Value

Untuk mengurutkan dictionary berdasarkan value (bukan key), kita buat list of tuples dengan format (value, key), lalu `sort`. Ini membalik urutan elemen sehingga sorting dilakukan berdasarkan value.

```
d = {'a': 10, 'b': 1, 'c': 22}

# Buat list (value, key)
lst = []
for key, val in d.items():
    lst.append((val, key))

# Sort descending
lst.sort(reverse=True)
print(lst)
# [(22, 'c'), (10, 'a'), (1, 'b')]

# Cetak hasil
for val, key in lst:
    print(f'{key}: {val}')
# c: 22
# a: 10
# b: 1
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

Buatlah sebuah program untuk mendapatkan nilai key, value, dan item dari sebuah dictionary dengan format tabel rapi.

Jawab:

```
def cek_semua_sama(t):  
    return len(set(t)) == 1  
  
if __name__ == "__main__":  
    tA = (90, 90, 90, 90)  
    print(f"tA= {tA}")  
    print()  
    print("Output")  
    print(cek_semua_sama(tA))
```

Output :

```
PS D:\Project\Kuliah\PrakAlPro-71251229> &  
tA= (90, 90, 90, 90)  
  
Output  
True
```

SOAL 2

Buatlah sebuah program untuk memetakan dua list menjadi satu dictionary.

Jawab:

```
data = ('Nama Lengkap Anda', '71251229', 'Kota, Provinsi')  
  
print(f"Data: {data}")  
print()  
  
# Unpack tuple  
nama, nim, alamat = data  
  
print(f"NIM      : {nim}")  
print(f"NAMA      : {nama}")
```

```

print(f"ALAMAT : {alamat}")
print()

# NIM per digit sebagai tuple
nim_tuple = tuple(nim)
print(f"NIM: {nim_tuple}")
print()

# Nama depan per karakter sebagai tuple
nama_depan = nama.split()[0]
nama_depan_tuple = tuple(nama_depan)
print(f"NAMA DEPAN: {nama_depan_tuple}")
print()

# Nama terbalik (split by spasi, dibalik) sebagai tuple
nama_terbalik = tuple(reversed(nama.split()))
print(f"NAMA TERBALIK: {nama_terbalik}")

```

Ouput:

```

PS D:\Project\Kuliah\PrakAlPro-71251229> & "C:/Users/M S I/AppData/Lo
Data: ('Ignatius Harya Nugraha', '71251229', 'Yogyakarta, DIY')

NIM    : 71251229
NAMA   : Ignatius Harya Nugraha
ALAMAT : Yogyakarta, DIY

NIM: ('7', '1', '2', '5', '1', '2', '2', '9')

NAMA DEPAN: ('I', 'g', 'n', 'a', 't', 'i', 'u', 's')

NAMA TERBALIK: ('Nugraha', 'Harya', 'Ignatius')

```

SOAL 3

Dengan menggunakan file mbox-short.txt, buat program yang membaca log email dan menyajikan histogram jumlah pesan per alamat email.

Jawab:

```

def hitung_distribusi_jam(nama_file):
    try:
        fhand = open(nama_file)
    except FileNotFoundError:
        print(f"File '{nama_file}' tidak bisa dibuka!")

```

```

        return

jam_counts = dict()
for line in fhand:
    if not line.startswith('From '):
        continue
    words = line.split()
    # Format: From email weekday month day HH:MM:SS year
    if len(words) < 6:
        continue
    waktu = words[5]          # contoh: '09:14:16'
    jam = waktu.split(':')[0] # ambil bagian jam saja
    jam_counts[jam] = jam_counts.get(jam, 0) + 1

fhand.close()

# Urutkan berdasarkan jam (sebagai list of tuples)
lst = sorted(jam_counts.items())

for jam, jumlah in lst:
    print(jam, jumlah)

if __name__ == "__main__":
    nama_file = input("Enter a file name: ")
    hitung_distribusi_jam(nama_file)

```

Output:

```

PS D:\Project\Kuliah\PrakAlPro-71251229> & "C:/Users/
Enter a file name: Minggu_11\mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1

```

Link repository saya: <https://github.com/Haigry/PrakAIPro-71251229>